

자동화 데이터 수집·요약 시스템 명세서 v2.0

Executive Summary

본 보고서는 경제시스템의 병목 가격결정력과 자본유입 동향을 매일 08:30에 텔레그램으로 전달하기 위한 자동화 데이터 파이프라인 설계 명세서이다. 모든 데이터 소스는 무료 공개 API만 사용한다.

한국은행 ECOS API, KOSIS 국가통계포털, KRX(한국거래소), Naver 뉴스 API, FRED 등 공신력 있는 무료 데이터 출처를 활용하여 거시·산업·시장·금융·물류·노동·가격·원자재·뉴스 등 다양한 지표를 수집한다. 수집된 원시 데이터는 APScheduler 기반으로 주기 수집 후 정제(결측치·정합성 검증) 단계를 거치고, SQLite 또는 PostgreSQL에 저장된다. 이후 이동평균·변동성·상관·회귀·변화점 탐지 등 통계 기법으로 요약 지표를 산출하며, 완성된 요약문과 Matplotlib 차트를 Python 코드로 생성하여 텔레그램 봇을 통해 매일 08:30에 전송한다.

파이프라인 요약:

- 데이터 소스: 한국은행 ECOS, KOSIS, KRX, FRED, Naver 뉴스, RSS, KIPRIS 등 (전부 무료)
- 파이프라인: APScheduler → 수집(재시도 포함) → 정제 → DB 저장 → 요약 연산 → 차트 생성 → 텔레그램 전송
- 요약 지표: 이동평균, 변동성, 상관분석, 회귀분석, 변화점 감지 등
- 텔레그램 전송: `python-telegram-bot` 라이브러리 활용
- 운영 모니터링: 수집 성공률, 지연 경보, 정확성·적시성·완전성 지표 모니터링
- 고도화: ML 예측, 자동이상탐지, 원인분석, 시각화 대시보드

1. 데이터 수집

1-1. 주요 데이터 항목 및 지표

- 거시경제 지표:** GDP, 산업생산지수, 소비자물가(CPI), 생산자물가, 고용률, 실업률 등. 한국은행 ECOS OpenAPI에서 국가계정·통화금융·물가·경기동향 통계 제공. KOSIS API를 통해 소비·투자·무역 통계를 JSON 형식으로 실시간 요청 가능.
- 산업·기업:** 제조업 출하·재고지수, 서비스업 동향조사, 주요 업종 생산·매출, 산업별 집중도 등. 통계청 공공데이터 활용.
- 시장·금융:** 코스피·코스닥 지수, 채권금리(국고채·회사채), 환율(원·달러, 원·엔) 등. KRX 공식 데이터 포털(data.krx.co.kr) 활용. 기준금리(BOK)는 한국은행 ECOS.
- 거래·소비:** 개인·외국인·기관의 주식 매수/매도, 코스피 거래대금, 소매판매지수(통계청) 등 소비활동 지표.
- 공급망·물류:** 항만 물동량, 컨테이너 운임지수. FRED의 글로벌 공급망 압력 지수(GSCPI) 활용. 이 지표는 병목현상을 보여주는 선행지표.
- 노동:** 고용률·실업률(KOSIS/통계청), 임금상승률, 구인구직 지수(워크넷). 인플레이션 시기 병목으로 인한 구인경색지수 추적.
- 가격 및 원자재:** 원자재 가격(두바이유: ECOS 902Y014, 금/금속: FRED), PMI(구매관리자지수), 환율. FRED의 곡물·에너지 시리즈 활용.
- 투자:** 국내외 FDI(외국인투자), 설비투자(KOSIS 설비투자지수), 기업 자금조달(유상증자, 회사채발행) 현황.
- 특허:** KIPRIS(특허청) OpenAPI를 통해 산업별 특허출원/등록 건수 확인. 신기술이나 디지털 전환 관련 특허증가율로 혁신도 측정.
- 뉴스:** Naver 뉴스 검색 API(당일 기사, 무료) 및 주요 경제지 RSS(연합뉴스, 매일경제, 한국경제)로 실시간 수집. 키워드 빈도, 감성 분석(금부정) 적용 가능.
- 검색 트렌드:** Google News RSS, pytrends(비공식 Google Trends)를 활용해 시장심리 지수 생성. KOSIS의 소비자심리지수도 활용.

1-2. 데이터 원천 및 API 목록 (무료 전용)

데이터 명	목적	지표 예시	API/출처	ECOS 통계표 코드 / 엔드포인트 예시	포맷	주기	인증
GDP	경제성장률 분	전분기비 성장률	한국은행 ECOS	<code>stat_id=100Y004</code>	JSON	분기	API Key

	석						
CPI	인 플 레 이 션 추 적	전년동 월比 상 승률	한국은행 ECOS	stat_id=901Y009	JSON	월 간	API Key
생산자 물가	공 급 측 물 가 분 석	전월比 상승률	한국은행 ECOS	stat_id=403Y014	JSON	월 간	API Key
원/달 러 환 율	환 율 동 향	매매기 준율	한국은행 ECOS	stat_id=731Y001	JSON	일 간	API Key
기준금 리	통 화 정 책 추 적	BOK 기준금 리	한국은행 ECOS	stat_id=722Y001	JSON	수 시	API Key
두바이 유	원 자 재 가 격	배럴당 USD	한국은행 ECOS	stat_id=902Y014	JSON	일 간	API Key
실업률	노 동 시 장 상 황	전체 실 업률	KOSIS	statId=J002	JSON	월 간	API Key
소매판 매지수	소 비 동 향	전월比 증감률	KOSIS	statId=K1001	JSON	월 간	API Key
설비투 자지수	투 자 동 향	전월比 증감률	KOSIS	statId=F002	JSON	월 간	API Key
코스피 지수	금 융 시 장 동 향	지수 종 가	KRX data.krx.co.kr	bld=dbms/MDC/STAT/standard/MDCSTAT00101	JSON	일 간 (거 래 일)	없음
코스닥 지수	중 소 형 시 장 동 향	지수 종 가	KRX data.krx.co.kr	bld=dbms/MDC/STAT/standard/MDCSTAT00101	JSON	일 간 (거 래 일)	없음

국제유가 (WTI)	에너지 가격	배럴당 USD	FRED	series_id=DCOILWTICO	JSON	일간	API Key
금 가격	안전 자산 동향	온스당 USD	FRED	series_id=GOLDFMGBD228NLBM	JSON	일간	API Key
미국 CPI	글로벌 인플레이	전년比 상승률	FRED	series_id=CPIAUCSL	JSON	월간	API Key
미국 기준금리	글로벌 금리	Fed Funds Rate	FRED	series_id=FEDFUNDS	JSON	월간	API Key
공급망 압력지수	공급망 병목	GSCPI	FRED	series_id=GSCPI	JSON	월간	API Key
뉴스 헤드라인	시장 감성 파악	경제 키워드 빈도	Naver 뉴스 API	https://openapi.naver.com/v1/search/news.json?query=경제	JSON	실시간	Client ID/Secret
뉴스 RSS	경제 이슈 수집	연합뉴스·매경 기사	RSS(무료)	https://news.google.com/rss/search?q=경제&hl=ko	XML	실시간	없음
특허출원 건수	혁신 동향 파악	산업별 특허 출원 수	KIPRIS Plus	/openapi/rest/TotalPatentSearchService/search	XML	월간	API Key
세계은행 GDP	국제 비교	국가별 GDP 성장률	World Bank API	https://api.worldbank.org/v2/country/KR/indicator/NY.GDP.MKTP.KD.ZG	JSON	연간	없음
OECD 지표	글로벌 비교	복합선 행지수 등	OECD.Stat API	https://stats.oecd.org/SDMX-JSON/data/MEI/KOR.CLI.M	JSON	월간	없음

우선순위 구분:

- ★ 필수: GDP, CPI, 생산자물가, 환율, 기준금리, 코스피, 실업률, 뉴스

- ☆ 권장: 두바이유, WTI, 금, 공급망압력지수, 특허, OECD 지표

1-3. 무료 API 인증키 발급처

API	발급처	URL
한국은행 ECOS	한국은행 경제통계시스템	https://ecos.bok.or.kr/api/
KOSIS	국가통계포털	https://kosis.kr/openapi/
FRED	St. Louis Fed	https://fred.stlouisfed.org/docs/api/
Naver 뉴스	Naver Developers	https://developers.naver.com
KIPRIS Plus	특허청	https://plus.kipris.or.kr

2. 데이터 파이프라인 설계

2-1. 스케줄링 (APScheduler)

Airflow 대신 **APScheduler**를 사용한다. 소규모 단독 서버 환경에 적합하며 별도 설치 부담이 없다.

```
# pip install apscheduler python-dotenv
from apscheduler.schedulers.blocking import BlockingScheduler
import logging

logging.basicConfig(level=logging.INFO,
                    format='%(asctime)s [% (levelname)s] %(message)s')

scheduler = BlockingScheduler(timezone="Asia/Seoul")

@scheduler.scheduled_job('cron', hour=8, minute=0)
def job_collect():
    """08:00 - 데이터 수집 및 DB 저장"""
    logging.info("데이터 수집 시작")
    collect_all_indicators()

@scheduler.scheduled_job('cron', hour=8, minute=20)
def job_summarize():
    """08:20 - 요약 지표 산출 및 차트 생성"""
    logging.info("요약 연산 시작")
    compute_summary()
    generate_charts()

@scheduler.scheduled_job('cron', hour=8, minute=30)
def job_send():
    """08:30 - 텔레그램 전송"""
    logging.info("텔레그램 전송 시작")
    send_telegram_report()

if __name__ == "__main__":
    scheduler.start()
```

2-2. 환경변수 관리 (.env)

API 키와 토큰은 절대 코드에 직접 포함하지 않는다.

```
# .env 파일 예시 (Git에 절대 커밋 금지 → .gitignore에 추가)
ECOS_API_KEY=your_ecos_key_here
KOSIS_API_KEY=your_kosis_key_here
FRED_API_KEY=your_fred_key_here
NAVER_CLIENT_ID=your_naver_client_id
NAVER_CLIENT_SECRET=your_naver_client_secret
KIPRIS_API_KEY=your_kipris_key_here
TELEGRAM_BOT_TOKEN=your_telegram_bot_token
TELEGRAM_CHAT_ID=your_chat_id
```

```
# config.py
from dotenv import load_dotenv
import os

load_dotenv()

ECOS_KEY      = os.getenv("ECOS_API_KEY")
KOSIS_KEY     = os.getenv("KOSIS_API_KEY")
FRED_KEY      = os.getenv("FRED_API_KEY")
NAVER_ID      = os.getenv("NAVER_CLIENT_ID")
NAVER_SECRET  = os.getenv("NAVER_CLIENT_SECRET")
KIPRIS_KEY    = os.getenv("KIPRIS_API_KEY")
TG_TOKEN      = os.getenv("TELEGRAM_BOT_TOKEN")
TG_CHAT_ID    = os.getenv("TELEGRAM_CHAT_ID")
```

2-3. 수집 모듈 (재시도·타임아웃 포함)

```
# collector.py
import requests
import logging
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry
from config import ECOS_KEY, KOSIS_KEY, FRED_KEY, NAVER_ID, NAVER_SECRET

def get_session():
    """재시도 로직이 포함된 HTTP 세션 반환"""
    session = requests.Session()
    retry = Retry(
        total=3,
        backoff_factor=2,          # 1초 → 2초 → 4초 대기
        status_forcelist=[429, 500, 502, 503, 504]
    )
    adapter = HTTPAdapter(max_retries=retry)
    session.mount("https://", adapter)
    session.mount("http://", adapter)
    return session

def fetch_ecos(stat_id: str, freq: str, start: str, end: str) -> list:
    """
    한국은행 ECOS API 호출
    freq: 'DD' (일), 'MM' (월), 'QQ' (분기)
    """
    url = (
        f"https://ecos.bok.or.kr/api/StatisticSearch/{ECOS_KEY}"
        f"/json/kr/1/100/{stat_id}/{freq}/{start}/{end}/"
    )
    session = get_session()
    try:
        res = session.get(url, timeout=10)
        res.raise_for_status()
        data = res.json()
        return data.get("StatisticSearch", {}).get("row", [])
    except Exception as e:
        logging.error(f"[ECOS] {stat_id} 수집 실패: {e}")
        return [] # 빈 리스트 반환 → 전일 데이터로 대체

def fetch_kosis(stat_id: str, start_period: str) -> list:
    """KOSIS OpenAPI 호출"""
    url = "https://kosis.kr/openapi/Param/statisticsList.do"
    params = {
        "method": "getList",
        "apiKey": KOSIS_KEY,
        "statId": stat_id,
        "prdSe": "M",
        "startPrdDe": start_period,
        "format": "json",
        "jsonVD": "Y"
    }
```

```

    }
    session = get_session()
    try:
        res = session.get(url, params=params, timeout=10)
        res.raise_for_status()
        return res.json()
    except Exception as e:
        logging.error(f"[KOSIS] {stat_id} 수집 실패: {e}")
        return []

def fetch_krx_index(market: str = "KOSPI") -> dict:
    """
    KRX 공식 데이터 포털에서 지수 수집
    market: 'KOSPI' or 'KOSDAQ'
    """
    url = "http://data.krx.co.kr/comm/bldAttendant/getJsonData.cmd"
    headers = {
        "User-Agent": "Mozilla/5.0",
        "Referer": "http://data.krx.co.kr"
    }
    payload = {
        "bld": "dbms/MDC/STAT/standard/MDCSTAT00101",
        "locale": "ko_KR",
        "mktId": "STK" if market == "KOSPI" else "KSQ",
    }
    session = get_session()
    try:
        res = session.post(url, data=payload, headers=headers, timeout=10)
        res.raise_for_status()
        return res.json()
    except Exception as e:
        logging.error(f"[KRX] {market} 수집 실패: {e}")
        return {}

def fetch_fred(series_id: str) -> list:
    """FRED API 호출 (무료, 키 필요)"""
    url = "https://api.stlouisfed.org/fred/series/observations"
    params = {
        "series_id": series_id,
        "api_key": FRED_KEY,
        "file_type": "json",
        "sort_order": "desc",
        "limit": 30
    }
    session = get_session()
    try:
        res = session.get(url, params=params, timeout=10)
        res.raise_for_status()
        return res.json().get("observations", [])
    except Exception as e:
        logging.error(f"[FRED] {series_id} 수집 실패: {e}")
        return []

def fetch_naver_news(query: str, display: int = 10) -> list:
    """Naver 뉴스 검색 API (당일 실시간, 무료)"""
    url = "https://openapi.naver.com/v1/search/news.json"
    headers = {
        "X-Naver-Client-Id": NAVER_ID,
        "X-Naver-Client-Secret": NAVER_SECRET
    }
    params = {
        "query": query,
        "display": display,
        "sort": "date"
    }
    session = get_session()
    try:
        res = session.get(url, headers=headers, params=params, timeout=10)
        res.raise_for_status()
        return res.json().get("items", [])

```

```

except Exception as e:
    logging.error(f"[NAVER] '{query}' 수집 실패: {e}")
    return []

def fetch_rss(url: str) -> list:
    """RSS 파싱 (완전 무료, 인증 불필요)"""
    import feedparser
    try:
        feed = feedparser.parse(url)
        return feed.entries[:10]
    except Exception as e:
        logging.error(f"[RSS] {url} 파싱 실패: {e}")
        return []

```

2-4. 저장소 설계 (SQLite / PostgreSQL)

소규모 운영 시 SQLite, 확장 필요 시 PostgreSQL을 사용한다.

```

# database.py
import sqlite3
import pandas as pd
from datetime import date

DB_PATH = "economic_data.db"

def init_db():
    """테이블 초기화"""
    conn = sqlite3.connect(DB_PATH)
    conn.execute("""
        CREATE TABLE IF NOT EXISTS indicators (
            id          INTEGER PRIMARY KEY AUTOINCREMENT,
            date        TEXT NOT NULL,
            indicator    TEXT NOT NULL,
            value       REAL,
            source      TEXT,
            unit        TEXT,
            updated     TEXT DEFAULT (datetime('now', 'localtime')),
            UNIQUE(date, indicator)
        )
    """)
    conn.execute("""
        CREATE TABLE IF NOT EXISTS collect_log (
            id          INTEGER PRIMARY KEY AUTOINCREMENT,
            run_date    TEXT,
            indicator    TEXT,
            status      TEXT,      -- 'success' or 'fail'
            message     TEXT,
            created     TEXT DEFAULT (datetime('now', 'localtime'))
        )
    """)
    conn.commit()
    conn.close()

def upsert_indicator(date_str: str, indicator: str, value: float,
                    source: str, unit: str = ""):
    """지표 저장 (중복 시 업데이트)"""
    conn = sqlite3.connect(DB_PATH)
    conn.execute("""
        INSERT INTO indicators (date, indicator, value, source, unit)
        VALUES (?, ?, ?, ?, ?)
        ON CONFLICT(date, indicator) DO UPDATE SET
            value=excluded.value,
            source=excluded.source,
            updated=datetime('now', 'localtime')
    """, (date_str, indicator, value, source, unit))
    conn.commit()
    conn.close()

def get_latest(indicator: str, n: int = 30) -> pd.DataFrame:

```

```

"""최근 n개 데이터 조회"""
conn = sqlite3.connect(DB_PATH)
df = pd.read_sql(
    "SELECT date, value FROM indicators WHERE indicator=? ORDER BY date DESC LIMIT ?",
    conn, params=(indicator, n)
)
conn.close()
return df.sort_values("date")

def get_previous_value(indicator: str) -> float:
    """전일 데이터 조회 (수집 실패 시 대체용)"""
    conn = sqlite3.connect(DB_PATH)
    cur = conn.execute(
        "SELECT value FROM indicators WHERE indicator=? ORDER BY date DESC LIMIT 1",
        (indicator,)
    )
    row = cur.fetchone()
    conn.close()
    return row[0] if row else None

def log_collect(indicator: str, status: str, message: str = ""):
    """수집 결과 로그 기록"""
    today = date.today().isoformat()
    conn = sqlite3.connect(DB_PATH)
    conn.execute(
        "INSERT INTO collect_log (run_date, indicator, status, message) VALUES (?, ?, ?, ?)",
        (today, indicator, status, message)
    )
    conn.commit()
    conn.close()

```

2-5. 정제 및 이상치 처리

```

# preprocessor.py
import pandas as pd
import numpy as np

def clean_series(df: pd.DataFrame, value_col: str = "value") -> pd.DataFrame:
    """
    결측치·이상치 처리
    1) 결측치: 직전값으로 선형보간
    2) 이상치: 3σ 초과값은 경계값으로 대체(winsorize)
    """
    df = df.copy().sort_values("date")
    df[value_col] = pd.to_numeric(df[value_col], errors="coerce")

    # 1. 결측치 보간
    df[value_col] = df[value_col].interpolate(method="linear").ffill().bfill()

    # 2. 이상치 처리 (3σ 룰)
    mean = df[value_col].mean()
    std = df[value_col].std()
    lower, upper = mean - 3 * std, mean + 3 * std
    outliers = (df[value_col] < lower) | (df[value_col] > upper)
    if outliers.any():
        import logging
        logging.warning(f"이상치 {outliers.sum()}건 감지 → 경계값으로 대체")
        df.loc[df[value_col] < lower, value_col] = lower
        df.loc[df[value_col] > upper, value_col] = upper

    return df

def compute_stats(df: pd.DataFrame, value_col: str = "value") -> dict:
    """이동평균, 변동성, 전일/전월 변화를 산출"""
    s = df[value_col].astype(float)
    return {
        "latest": round(s.iloc[-1], 4) if len(s) > 0 else None,
        "prev": round(s.iloc[-2], 4) if len(s) > 1 else None,
        "change_pct": round((s.iloc[-1] / s.iloc[-2] - 1) * 100, 2) if len(s) > 1 else None,
    }

```



```

"ma5":          round(s.rolling(5).mean().iloc[-1], 4) if len(s) >= 5 else None,
"ma20":         round(s.rolling(20).mean().iloc[-1], 4) if len(s) >= 20 else None,
"volatility": round(s.rolling(20).std().iloc[-1], 4) if len(s) >= 20 else None,
}

```

3. 요약 지표 산출 명세

```

# summarizer.py
from database import get_latest
from preprocessor import clean_series, compute_stats
import pandas as pd

INDICATORS = {
    "CPI":          {"label": "소비자물가지수", "unit": "%"},
    "PPI":          {"label": "생산자물가지수", "unit": "%"},
    "USD_KRW":      {"label": "원/달러 환율", "unit": "원"},
    "BASE_RATE":    {"label": "기준금리", "unit": "%"},
    "KOSPI":        {"label": "코스피", "unit": "pt"},
    "UNEMPLOYMENT": {"label": "실업률", "unit": "%"},
    "DUBAI_OIL":    {"label": "두바이유", "unit": "USD/bbl"},
    "WTI":          {"label": "WTI 국제유가", "unit": "USD/bbl"},
    "GOLD":         {"label": "금 가격", "unit": "USD/oz"},
}

def build_summary() -> dict:
    """모든 지표의 최신 통계 산출"""
    result = {}
    for key, meta in INDICATORS.items():
        df = get_latest(key, n=30)
        if df.empty:
            result[key] = {"label": meta["label"], "unit": meta["unit"], "error": "데이터 없음"}
            continue
        df = clean_series(df)
        stats = compute_stats(df)
        result[key] = {**meta, **stats}
    return result

```

4. 차트 생성 명세

```

# chart_generator.py
import matplotlib
matplotlib.use("Agg") # 헤드리스 서버 환경 필수
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
import io
from database import get_latest
from preprocessor import clean_series

# -----
# 한글 폰트 설정 (서버 배포 시 반드시 설치 필요)
# sudo apt-get install fonts-nanum (Ubuntu)
# -----

def set_korean_font():
    candidates = [
        "/usr/share/fonts/truetype/nanum/NanumGothic.ttf",
        "/usr/share/fonts/nanum/NanumGothic.ttf",
        "NanumGothic",
        "AppleGothic", # macOS
        "Malgun Gothic", # Windows
    ]
    for font in candidates:
        try:
            fm.findfont(fm.FontProperties(fname=font))
            plt.rcParams["font.family"] = fm.FontProperties(fname=font).get_name()

```

```

        return
    except Exception:
        continue
plt.rcParams["font.family"] = "sans-serif"

set_korean_font()
plt.rcParams["axes.unicode_minus"] = False

def create_multi_chart(indicators: list) -> io.BytesIO:
    """
    여러 지표를 2열 격자 차트로 생성
    indicators: [("KOSPI", "코스피"), ("USD_KRW", "원/달러 환율"), ...]
    반환: PNG BytesIO (텔레그램 직접 전송 가능)
    """
    n = len(indicators)
    cols = 2
    rows = (n + 1) // cols
    fig, axes = plt.subplots(rows, cols, figsize=(12, rows * 3))
    axes = axes.flatten()

    for i, (key, label) in enumerate(indicators):
        df = get_latest(key, n=60)
        if df.empty:
            axes[i].text(0.5, 0.5, "데이터 없음", ha="center", va="center")
            axes[i].set_title(label)
            continue
        df = clean_series(df)
        x = pd.to_datetime(df["date"])
        y = df["value"].astype(float)

        axes[i].plot(x, y, color="#1E88E5", linewidth=1.5)
        axes[i].fill_between(x, y, alpha=0.1, color="#1E88E5")
        axes[i].set_title(label, fontsize=11, fontweight="bold")
        axes[i].grid(True, alpha=0.3, linestyle="--")
        axes[i].tick_params(axis="x", rotation=30, labelsize=7)

    # 빈 서브플롯 숨기기
    for j in range(i + 1, len(axes)):
        axes[j].set_visible(False)

    fig.suptitle("주요 경제지표 추이", fontsize=14, fontweight="bold", y=1.01)
    plt.tight_layout()

    buf = io.BytesIO()
    plt.savefig(buf, format="png", dpi=150, bbox_inches="tight")
    buf.seek(0)
    plt.close(fig)
    return buf

```

5. 텔레그램 전송 명세

```

# telegram_sender.py
import logging
from telegram import Bot
from telegram.error import TelegramError
from config import TG_TOKEN, TG_CHAT_ID
from summarizer import build_summary
from chart_generator import create_multi_chart
from database import get_latest
import pandas as pd
import io

def format_change(val) -> str:
    if val is None:
        return "N/A"
    arrow = "▲" if val > 0 else ("▼" if val < 0 else "➡")

```

```

        return f"{arrow} {val:+.2f}%"

def build_message(summary: dict) -> str:
    lines = ["🇰🇷 *오늘의 경제지표 요약*\n"]
    for key, info in summary.items():
        if "error" in info:
            lines.append(f"• {info['label']}: ⚠️ {info['error']}")
        else:
            latest = f"{info['latest']:.2f} {info['unit']}" if info.get("latest") else "N/A"
            change = format_change(info.get("change_pct"))
            lines.append(f"• {info['label']}: `{latest}` {change}")
    return "\n".join(lines)

def send_report():
    bot = Bot(token=TG_TOKEN)

    # 1. 요약 텍스트 전송
    summary = build_summary()
    message = build_message(summary)
    try:
        bot.send_message(
            chat_id=TG_CHAT_ID,
            text=message,
            parse_mode="Markdown"
        )
    except TelegramError as e:
        logging.error(f"텍스트 전송 실패: {e}")

    # 2. 차트 이미지 전송
    indicators = [
        ("KOSPI", "코스피"),
        ("USD_KRW", "원/달러 환율"),
        ("CPI", "소비자물가(CPI)"),
        ("DUBAI_OIL", "두바이유"),
    ]
    try:
        chart_buf = create_multi_chart(indicators)
        bot.send_photo(
            chat_id=TG_CHAT_ID,
            photo=chart_buf,
            caption="📈 주요 지표 추이 (최근 60일)"
        )
    except TelegramError as e:
        logging.error(f"차트 전송 실패: {e}")

    # 3. CSV 로우데이터 전송 (선택)
    try:
        rows = []
        for key in summary:
            df = get_latest(key, n=5)
            df["indicator"] = key
            rows.append(df)
        if rows:
            combined = pd.concat(rows)
            csv_buf = io.BytesIO(combined.to_csv(index=False).encode("utf-8-sig"))
            bot.send_document(
                chat_id=TG_CHAT_ID,
                document=csv_buf,
                filename="daily_indicators.csv",
                caption="📄 원시 데이터 (CSV)"
            )
    except TelegramError as e:
        logging.error(f"CSV 전송 실패: {e}")

```

텔레그램 메시지 포맷 예시

🇰🇷 *오늘의 경제지표 요약*

- 소비자물가지수: `104.20` % ⬆️ +0.30%

- 생산자물가지수: `112.50 %` ▲ +0.50%
- 원/달러 환율: `1,320.50 원` ▲ +0.20%
- 기준금리: `3.50 %` ➡ +0.00%
- 코스피: `2,480.30 pt` ▲ +0.40%
- 실업률: `2.80 %` ▼ -0.10%
- 두바이유: `82.40 USD/bbl` ▼ -0.80%

6. 운영·모니터링·로깅

```
# monitor.py
import sqlite3
from datetime import date
import logging
from config import TG_TOKEN, TG_CHAT_ID
from telegram import Bot

DB_PATH = "economic_data.db"

def check_data_quality() -> dict:
    """
    데이터 품질 지표 산출
    - 완전성: 오늘 수집된 지표 수 / 전체 지표 수
    - 수집 성공률: 오늘 성공 / 전체 시도
    """
    today = date.today().isoformat()
    conn = sqlite3.connect(DB_PATH)

    total_indicators = 9 # INDICATORS 딕셔너리 항목 수
    collected = conn.execute(
        "SELECT COUNT(DISTINCT indicator) FROM indicators WHERE date=?", (today,)
    ).fetchone()[0]

    logs = conn.execute(
        "SELECT status, COUNT(*) FROM collect_log WHERE run_date=? GROUP BY status", (today,)
    ).fetchall()
    conn.close()

    log_dict = dict(logs)
    success = log_dict.get("success", 0)
    fail = log_dict.get("fail", 0)
    total = success + fail

    return {
        "completeness": round(collected / total_indicators * 100, 1) if total_indicators else 0,
        "success_rate": round(success / total * 100, 1) if total else 0,
        "fail_count": fail,
        "collected_count": collected,
    }

def alert_if_fail():
    """수집 실패 시 텔레그램 관리자 알림"""
    quality = check_data_quality()
    if quality["fail_count"] > 0 or quality["completeness"] < 80:
        bot = Bot(token=TG_TOKEN)
        msg = (
            f"⚠️ *데이터 수집 경보*\n"
            f"• 완전성: {quality['completeness']}%\n"
            f"• 수집 성공률: {quality['success_rate']}%\n"
            f"• 실패 건수: {quality['fail_count']}건"
        )
        bot.send_message(chat_id=TG_CHAT_ID, text=msg, parse_mode="Markdown")
        logging.warning(f"품질 경보 발송: {quality}")
```

7. 파이프라인 통합 실행

```

# main.py
import logging
from database import init_db
from collector import (fetch_ecos, fetch_kosis, fetch_krx_index,
                       fetch_fred, fetch_naver_news)
from database import upsert_indicator, log_collect, get_previous_value
from summarizer import build_summary
from chart_generator import create_multi_chart
from telegram_sender import send_report
from monitor import alert_if_fail
from datetime import date

logging.basicConfig(
    filename="pipeline.log",
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s"
)

TODAY = date.today().strftime("%Y%m%d")

def safe_store(indicator: str, value, source: str, unit: str = ""):
    """수집값 저장 실패 시 전일값으로 대체"""
    if value is None:
        value = get_previous_value(indicator)
        logging.warning(f"[{indicator}] 전일값으로 대체: {value}")
        log_collect(indicator, "fail", "전일값 대체")
    else:
        log_collect(indicator, "success")
    if value is not None:
        upsert_indicator(date.today().isoformat(), indicator, float(value), source, unit)

def collect_all_indicators():
    # — 한국은행 ECOS —————
    # CPI (소비자물가)
    rows = fetch_ecos("901Y009", "MM", "202301", TODAY[:6])
    val = float(rows[-1]["DATA_VALUE"]) if rows else None
    safe_store("CPI", val, "ECOS:901Y009", "%")

    # 원/달러 환율
    rows = fetch_ecos("731Y001", "DD", "20240101", TODAY)
    val = float(rows[-1]["DATA_VALUE"]) if rows else None
    safe_store("USD_KRW", val, "ECOS:731Y001", "원")

    # 두바이유
    rows = fetch_ecos("902Y014", "DD", "20240101", TODAY)
    val = float(rows[-1]["DATA_VALUE"]) if rows else None
    safe_store("DUBAI_OIL", val, "ECOS:902Y014", "USD/bbl")

    # 기준금리
    rows = fetch_ecos("722Y001", "MM", "202301", TODAY[:6])
    val = float(rows[-1]["DATA_VALUE"]) if rows else None
    safe_store("BASE_RATE", val, "ECOS:722Y001", "%")

    # — KRX —————
    krx = fetch_krx_index("KOSPI")
    # KRX 응답 구조에 따라 파싱 필요 (실제 응답 확인 후 키 수정)
    val = None
    if krx and "output" in krx:
        try:
            val = float(krx["output"][0].get("CLSPRC", "").replace(", ", ""))
        except Exception:
            pass
    safe_store("KOSPI", val, "KRX", "pt")

    # — FRED —————
    # WTI 국제유가
    obs = fetch_fred("DCOILWTICO")
    val = float(obs[0]["value"]) if obs and obs[0]["value"] != "." else None
    safe_store("WTI", val, "FRED:DCOILWTICO", "USD/bbl")

```

```

# 금 가격
obs = fetch_fred("GOLDPMGBD228NLBM")
val = float(obs[0]["value"]) if obs and obs[0]["value"] != "." else None
safe_store("GOLD", val, "FRED:GOLDPMGBD228NLBM", "USD/oz")

# --- KOSIS ---
# 실업률
rows = fetch_kosis("J002", "202301")
# KOSIS 응답 파싱 (실제 응답 확인 후 키 수정)
val = None
if isinstance(rows, list) and rows:
    try:
        val = float(rows[-1].get("DT", 0))
    except Exception:
        pass
safe_store("UNEMPLOYMENT", val, "KOSIS:J002", "%")

logging.info("전체 수집 완료")

if __name__ == "__main__":
    init_db()
    collect_all_indicators()
    summary = build_summary()
    send_report()
    alert_if_fail()

```

8. 파이프라인 아키텍처 다이어그램

```

graph TD
    subgraph "무료 데이터 소스"
        A1[한국은행 ECOS API]
        A2[KOSIS 국가통계]
        A3[KRX 거래소]
        A4[FRED 미연준]
        A5[Naver 뉴스 API]
        A6[RSS 피드]
        A7[KIPRIS 특허청]
    end

    subgraph "ETL 파이프라인"
        B1[APScheduler 08:00]
        B2[수집 모듈 재시도·타임아웃]
        B3[정제·이상치처리]
        B4[SQLite / PostgreSQL]
        B5[전일값 대체 Fallback]
    end

    subgraph "요약·분석"
        C1[이동평균·변동성·변화율]
        C2[이상치 필터링 3σ]
        C3[요약 딕셔너리 생성]
    end

    subgraph "결과 전송 08:30"
        D1[Matplotlib 차트 생성]
        D2[텔레그램 봇 전송]
        D3[CSV 첨부 선택]
    end

    subgraph "모니터링"
        E1[수집 성공률 체크]
        E2[품질 경보 텔레그램]
        E3[pipeline.log]
    end

    A1 & A2 & A3 & A4 & A5 & A6 & A7 --> B2

```

```
B1 --> B2
B2 -- 수집 성공 --> B3
B2 -- 수집 실패 --> B5
B5 --> B3
B3 --> B4
B4 --> C1 --> C2 --> C3
C3 --> D1 --> D2
C3 --> D3 --> D2
B2 --> E3
B4 --> E1 --> E2
```

9. 프로젝트 구조 및 의존성

디렉터리 구조

```
project/
├── .env                # API 키 (Git 제외)
├── .gitignore
├── main.py             # 진입점
├── scheduler.py        # APScheduler 실행
├── config.py           # 환경변수 로드
├── collector.py        # API 수집 모듈
├── preprocessor.py     # 정제·이상치 처리
├── database.py         # SQLite CRUD
├── summarizer.py       # 지표 요약 산출
├── chart_generator.py  # Matplotlib 차트
├── telegram_sender.py  # 텔레그램 전송
├── monitor.py          # 품질 모니터링
├── economic_data.db    # SQLite DB (자동 생성)
├── pipeline.log        # 실행 로그 (자동 생성)
└── requirements.txt
```

requirements.txt

```
requests>=2.31.0
pandas>=2.0.0
numpy>=1.24.0
matplotlib>=3.7.0
python-telegram-bot>=20.0
apscheduler>=3.10.0
python-dotenv>=1.0.0
feedparser>=6.0.0
ruptures>=1.1.7
```

서버 한글 폰트 설치 (Ubuntu 기준)

```
sudo apt-get update
sudo apt-get install -y fonts-nanum
fc-cache -fv
```

10. 고도화 제안

- **변화점 탐지:** Python `ruptures` 라이브러리로 급격한 추세변화(예: 물가 급등 시작) 자동 감지 및 알람
 - **머신러닝 예측:** Prophet 또는 ARIMA로 CPI·환율 단기 예측 후 예측 편차를 경고 신호로 활용
 - **감성 분석:** Naver 뉴스·RSS 텍스트에 KR-FinBERT 등 한국어 모델 적용하여 시장 심리 지수 산출
 - **시각화 대시보드:** Grafana(무료) + SQLite 연동으로 운영자용 실시간 대시보드 구축
 - **사용자 피드백:** 텔레그램 봇에 인라인 버튼을 추가하여 특정 지표 상세 조회 기능 제공
-

본 명세서는 모든 데이터 소스를 무료 공개 API로 한정하며, 유료 서비스(Bloomberg 등)는 포함하지 않는다.